

TECHNICAL DOCUMENT

Frontend Design Requirements

Architecture, component structure, state management, API integration, and performance strategy for the Product Management SPA.

PROJECT

Product Management

FRAMEWORK

React 19 + TypeScript 6

DATE

August 2026

Table of Contents

1 Project Structure

1.1 Folder Organisation

1.2 Feature-Slice Pattern

1.3 Shared Utilities

2 UI Layout

2.1 Application Shell

2.2 Routing & Navigation

2.3 Page Layouts

3 Technology Stack

3.1 UI Framework

3.2 State Management

3.3 API Client

3.4 File Uploader

4 API & Data Handling

4.1 Base Query & Error Normalisation

4.2 Input Validation

4.3 Response Handling & Cache Tags

4.4 Concurrent Edit (ETag / If-Match)

5 Performance

5.1 Code Splitting & Lazy Loading

5.2 Optimistic Updates

5.3 Debouncing & Prefetching

5.4 Error Handling Strategy

1 Project Structure

1.1 Folder Organisation

The frontend follows a **feature-slice** architecture. Each domain feature owns its own components, API hooks, types, and tests. Cross-feature code lives in `shared/`. There is no monolithic component folder.

```
src/ └─ app/ │ └─ router.tsx # route definitions with lazy imports │ └─
store.ts # Redux store + RTK Query wiring │ └─ features/ │ └─ products/ │ │ └─
api.ts # RTK Query endpoints for products & items │ │ └─ types.ts # Product,
ProductItem, ProductListItem │ │ └─ ProductListPage.tsx │ │ └─
ProductDetailPage.tsx │ │ └─ ProductForm.tsx │ │ └─ VariantsTable.tsx │ │ └─
ImageUploader.tsx │ │ └─ *.test.tsx │ └─ categories/ │ └─ api.ts │ └─ types.ts
│ └─ CategoryListPage.tsx │ └─ CategoryForm.tsx │ └─ buildCategoryTree.ts │ └─
*.test.tsx │ └─ shared/ │ └─ components/ │ │ └─ AppLayout.tsx # shell: sidebar
+ header + content │ │ └─ ErrorBoundary.tsx # catches unhandled render errors │
└─ hooks/ │ │ └─ useDebounceValue.ts │ └─ lib/ │ └─ apiBase.ts # RTK Query
createApi base │ └─ axiosClient.ts # Axios instance + interceptors │ └─
errors.ts # AppError normalisation │ └─ etagStore.ts # in-memory ETag cache │ └─
resolveImageUrl.ts │ └─ mocks/ │ └─ handlers.ts # MSW mock handlers (tests only)
│ └─ server.ts │ └─ main.tsx # entry point: Provider + RouterProvider
```

1.2 Feature-Slice Pattern

Each feature directory is self-contained. A feature exposes only what other features need (typically its route component) — internal sub-components, hooks, and utilities stay private to the feature. This keeps coupling low and makes it straightforward to add, remove, or refactor a feature without touching unrelated code.

What lives in a feature

- Page component(s)
- RTK Query API slice (`api.ts`)
- TypeScript interfaces (`types.ts`)
- Sub-components (forms, tables)
- Co-located unit/component tests

What lives in shared

- App shell layout
- Error boundary
- Custom hooks
- Axios instance & error utilities
- RTK Query base configuration

1.3 Shared Utilities

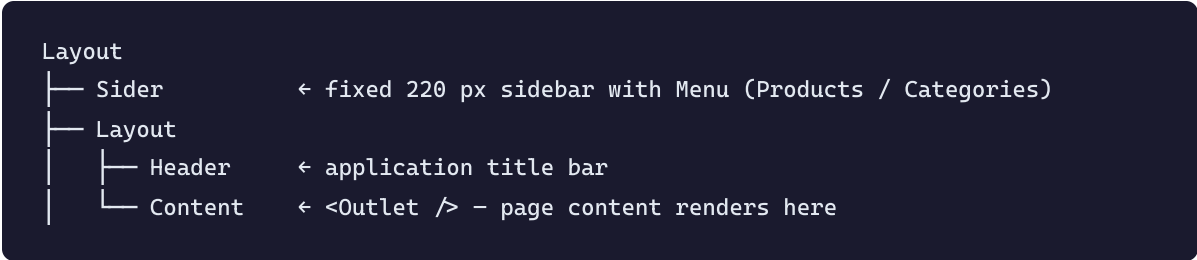
File	Responsibility
<code>apiBase.ts</code>	Single <code>createApi</code> instance shared by all feature API slices
<code>axiosClient.ts</code>	Axios instance with base URL, response interceptor (ETag capture), error interceptor
<code>errors.ts</code>	Converts Axios errors → <code>AppError</code> (status, message, fieldErrors, raw)
<code>etagStore.ts</code>	In-memory <code>Map<url, etag></code> for optimistic concurrency
<code>resolveImageUrl.ts</code>	Turns relative upload paths into fully qualified API-origin URLs
<code>useDebouncedValue.ts</code>	Generic debounce hook used for search input (350 ms delay)

2

UI Layout

2.1 Application Shell

The entire application is wrapped in `AppLayout`, an Ant Design `Layout` component that provides a fixed left sidebar, a top header bar, and a scrollable content area. The sidebar highlights the active route automatically using React Router's `useLocation` hook.



2.2 Routing & Navigation

All routes are defined in `app/router.tsx` and are lazy-loaded.

Path	Component	Purpose
/	ProductListPage	Default redirect — product list
/products	ProductListPage	Search, filter, paginate products
/products/new	ProductDetailPage	Create a new product
/products/:id	ProductDetailPage	Edit existing product & manage items
/categories	CategoryListPage	Manage categories (CRUD inline)

Every page component is wrapped with a `withSuspense()` helper that renders an Ant Design `<Spin />` spinner as the Suspense fallback while the JavaScript chunk downloads.

2.3 Page Layouts

Product List Page

- Search bar + category filter + "New Product" button in a flex toolbar
- Ant Design `Table` with image thumbnail, name, price range, stock, status, and edit action
- Cursor-based pagination controls (Previous / Next) with page indicator
- Inline `Result` error state with Retry button on API failure

Product Detail Page

- `ProductForm` — core product fields (name, slug, category, brand, description)
- `ImageUploader` — drag-and-drop / click image upload with preview
- `VariantsTable` — list of product items with stock +/- controls and delete
- Inline "Add Variant" form below the table

Category List Page

- Ant Design `Table` rendering a flat list with parent category name column
- Inline `CategoryForm` drawer/modal for create and edit
- Hierarchical `TreeSelect` for parent category assignment

3

Technology Stack Components

3.1 UI Framework — Ant Design 6

Ant Design (antdv6) is used as the sole UI component library. It provides a comprehensive set of production-ready components, eliminating the need for a separate CSS framework like Tailwind or Material UI.

Component	Used for
Layout / Sider / Header	Application shell
Table	Product list, category list, variants table
Form / Input / Select / TreeSelect	Product & category forms
Upload	Image upload with preview
Modal / Popconfirm	Conflict dialog, delete confirmation
Result	API error state, error boundary display
Spin / Button / Space / message	Loading, actions, toasts

3.2 State Management — Redux Toolkit + RTK Query

Redux Toolkit (RTK) is used for global state. In practice, the only global state is server data — managed entirely by **RTK Query**, the data-fetching and caching layer bundled with RTK. There are no hand-written Redux slices.

```
// store.ts
export const store = configureStore({
  reducer: {
    [api.reducerPath]: api.reducer, // all server state lives here
  },
  middleware: (getDefault) =>
    getDefault().concat(api.middleware), // adds caching, invalidation, polling
});
```

RTK Query handles request deduplication, background refetching, cache tag invalidation, optimistic updates, and loading/error states — removing the need for any manual `useEffect`-based data fetching.

3.3 API Client — Axios + RTK Query custom baseQuery

Axios v1 is used as the HTTP client. Rather than using RTK Query's default `fetchBaseQuery`, a custom `axiosBaseQuery` wraps Axios so that all requests flow through the shared Axios instance (which carries interceptors for ETag capture and error normalisation).

```
// axiosClient.ts - response interceptor
axios.interceptors.response.use((response) => {
  const etag = response.headers['etag'];
  if (etag) etagStore.set(response.config.url!, etag); // cache ETag per URL
  return response;
});

// apiBase.ts - custom baseQuery fed to createApi
const axiosBaseQuery: BaseQueryFn = async ({ url, method, data, params }) => {
  try {
    const result = await axiosClient({ url, method, data, params });
    return { data: result.data };
  } catch (err) {
    return { error: toAppError(err) }; // normalise all errors to AppError
  }
};
```

3.4 File Uploader — Ant Design Upload + RTK Query mutation

Image upload is handled by the `ImageUploader` component using Ant Design's `<Upload>` with a **custom request handler**. The default upload behaviour is suppressed; instead, the component manually builds a `FormData` object and dispatches an RTK Query mutation.

Concern	Implementation
Allowed types	JPEG, PNG, WEBP — validated in <code>beforeUpload()</code>
Max size	5 MB — validated in <code>beforeUpload()</code> before upload starts
Transport	<code>multipart/form-data</code> via <code>uploadImage</code> RTK Query mutation
Preview	Inline image rendered from resolved API URL; hidden if URL is null
Fallback	<code>onError</code> handler switches to placeholder SVG on broken images
Delete	<code>deleteImage</code> mutation; cache is invalidated to refresh the product

4

API & Data Handling

4.1 Base Query & Error Normalisation

All API calls go through a single Axios instance. Errors from any endpoint are converted into a consistent `AppError` shape by `toAppError()` in `errors.ts`, which parses the backend's RFC-9457 `ProblemDetails` response body.

```
interface AppError {
  status:      number;      // HTTP status code
  message:     string;      // human-readable title
  fieldErrors: FieldError[]; // [{propertyName, errorMessage}] for 400 validation
  traceId:     string | null;
  raw:         unknown;     // original response body (for non-standard shapes)
}

interface FieldError {
  propertyName: string; // e.g. "name", "slug"
  errorMessage: string;
}
```

4.2 Input Validation

Validation operates on two levels: client-side (Ant Design Form rules) and server-side (FluentValidation errors mapped back to form fields).

Client-side — Ant Design Form rules

- **Name:** required, max 200 characters
- **Slug:** required (create only), must match `/^[a-z0-9-]+$ /`
- **Category:** required dropdown selection
- **SKU:** required, max 64 characters
- **Price:** required, ≥ 0
- **Qty in stock:** required integer, ≥ 0

Server-side — field error mapping

When the API returns `400 Bad Request` with `fieldErrors`, the component maps them onto Ant Design form fields so validation messages appear inline under the relevant inputs:

```
// ProductForm.tsx
const fieldErrors = (err as AppError).fieldErrors ?? [];
form.setFields({
```

```

fieldErrors.map(({ propertyName, errorMessage }) => ({
  name: propertyName.charAt(0).toLowerCase() + propertyName.slice(1),
  errors: [errorMessage],
}))
);

```

4.3 Response Handling & Cache Tags

RTK Query uses a tag-based cache invalidation system. Every mutation declares which tags it invalidates, and every query declares which tags it provides.

Mutation	Tags invalidated	Effect
<code>createProduct</code>	<code>ProductList</code>	Refetches the product list
<code>updateProduct</code>	<code>Product(id)</code> , <code>ProductList</code>	Refetches the product and list
<code>deleteProduct</code>	<code>Product(id)</code> , <code>ProductList</code>	Removes from list
<code>createProductItem</code>	<code>Product(productId)</code>	Refreshes detail page
<code>deleteProductItem</code>	<code>Product(productId)</code>	Refreshes detail page
<code>uploadImage / deleteImage</code>	<code>Product(id)</code> , <code>ProductList</code>	Updates image in list & detail
<code>adjustStock</code>	<code>Product(productId)</code>	Refreshes stock count (after optimistic)

4.4 Concurrent Edit — ETag / If-Match

To prevent lost updates when two users edit the same product simultaneously, the app implements **optimistic concurrency** using HTTP ETags.

1. `GET /products/:id` — the Axios response interceptor stores the `ETag` header in `etagStore`
2. `PUT /products/:id` — the mutation reads the stored ETag and sends it as an `If-Match` header
3. If the server responds with `409 Conflict`, `ProductForm` shows a modal: "This product was changed by someone else" with a **Reload latest** button
4. Clicking Reload invalidates the cache, fetches the fresh version, and closes the modal

5

Performance

5.1 Code Splitting & Lazy Loading

All three page-level components are code-split using `React.lazy()`. Vite automatically produces a separate JS chunk per lazy import. The browser only downloads a page's chunk when the user first navigates to that route.

```
// app/router.tsx
const ProductListPage = lazy(() => import('../features/products/ProductListPage'));
const ProductDetailPage = lazy(() => import('../features/products/ProductDetailPage'));
const CategoryListPage = lazy(() => import('../features/categories/CategoryListPage'));

// Each is wrapped in Suspense with a loading spinner
function withSuspense(Component: ComponentType) {
  return (
    <Suspense fallback=<Spin size="large" />>
      <Component />
    </Suspense>
  );
}
```

Product thumbnail images in the list use the browser's native lazy loading:

```
<img src={...} loading="lazy" style={{ width: 40, height: 40, objectFit: 'cover' }} />
```

5.2 Optimistic Updates — Stock Adjustment

The `adjustStock` mutation applies an **optimistic update**: the UI increments or decrements the stock count immediately, without waiting for the server response. If the server rejects the request (e.g. insufficient stock), RTK Query automatically rolls back the cache to its previous state.

```
adjustStock: builder.mutation({
  async onQueryStarted({ productId, itemId, delta }, { dispatch, queryFulfilled }) {
    const patch = dispatch(
      productsApi.util.updateQueryData('getProduct', productId, (draft) => {
        const item = draft.items.find((i) => i.id === itemId);
        if (item) item.qtyInStock += delta;
      })
    );
  }
});
```

```
    try {
      await queryFulfilled;
    } catch {
      patch.undo(); // roll back on server error
    }
  },
})
})
```

5.3 Debouncing & Prefetching

Search debounce

The product search input is debounced by **350 ms** via `useDebounceValue`. The API query is only fired after the user pauses typing, preventing a network request on every keystroke.

```
const debouncedQ = useDebounceValue(q, 350);
const { data } = useListProductsQuery({ q: debouncedQ || undefined, ... });
```

Hover prefetch

When the user hovers over a product row in the list, the detail query is prefetched silently in the background. By the time the user clicks "Edit", the data is already in the RTK Query cache — the detail page appears instantly.

```
const prefetchProduct = productsApi.usePrefetch('getProduct');

// on table row hover:
onMouseEnter: () => prefetchProduct(record.id)
```

5.4 Error Handling Strategy

The app uses three layers of error handling, from coarse to fine:

Layer	Mechanism	Handles
Global	<code>ErrorBoundary</code> (class component)	Uncaught render errors — shows a full-page error Result with Reload
Page	RTK Query <code>isError</code> flag + Ant Design <code>Result</code>	Network failure / API unreachable — inline error with Retry button
Component	<code>try/catch</code> around mutations + <code>message.error()</code>	Mutation failures (save, delete, stock adjust) — toast notification

Layer	Mechanism	Handles
Form field	<code>form.setFields(fieldErrors)</code>	Server-side 400 validation — inline messages under each input
Conflict	409 Conflict → Modal dialog	Concurrent edit detection — user prompted to reload latest version
Image	<code>img onError</code> fallback	Broken image URLs — replaced with placeholder SVG

The stock endpoint returns a non-standard 409 body (`{ availableQuantity }` instead of `ProblemDetails`). The `raw` field on `AppError` preserves this original response so the component can surface a specific message: *"Only N left — you requested more than that."*